1 Анализ задачи

1.1 Постановка задачи

Разработать электронный образовательный ресурс по электронной платформе «Ардуино».

Создание программной среды, выполняющей функции доступной виртуальной лаборатории на базе Arduino UNO [1]. Продукт позволит начинающим пользователям безопасно осваивать азы робототехники, конструировать электрические цепи на виртуальной макетной плате и программировать микроконтроллер для решения прикладных инженерных задач без риска повреждения реального оборудования.

Целевой аудиторией приложения являются пользователи старше 12 лет, в том числе школьники, студенты и энтузиасты, самостоятельно изучающие схемотехнику. Программа предоставляет надежную и интерактивную платформу для ознакомления с принципами функционирования платы Arduino UNO, сборки схем и тестирования написанного кода в безопасных виртуальных условиях.

Периодичность использования определяется потребностями пользователя (при выполнении учебных заданий или реализации технических проектов).

В качестве примера можно привести веб-платформу https://wokwi.com/arduino, она обладает схожим функционалом – позволяет проектировать электрические схемы и отлаживать программный код. В отличие от аналога, разрабатываемый продукт функционирует в автономном режиме (без подключения к сети Интернет) и исключает задержки, связанные с очередью на компиляцию.

Входная информация: топология собранной пользователем электрической цепи – список проводных соединений между контактами макетной платы и радиодеталей; пользовательский код, введённый в текстовый редактор; набор требований текущего уровня.

Постоянная информация: правила работы каждого типа радиодетали; карта контактных точек платы с ролями выводов; правила синтаксического разбора пользовательского кода и его преобразования в Lua [2].

Выходная информация: логический уровень каждого контакта схемы; признак корректности или ошибки сборки цепи или кода; итоговый результат проверки задания уровня (выполнено/не выполнено). Формат представления – графическое окно программы с отображением состояния цепи и всплывающими диагностическими окнами.

1.2 Инструменты разработки

Для реализации симулятора применяется интегрированная среда Microsoft Visual Studio 2022, а также мультимедийная библиотека SFML [3] и язык C++ [4, 5, 6], встроенный скриптовый язык Lua и связующая библиотека sol2 [7].

1 Microsoft Visual Studio 2022 выступает в роли производительной IDE для C++, предлагающей мощный встроенный отладчик, соответствие современным стандартам языка и эффективные средства профилирования кода.

2 SFML [8] представляет собой кроссплатформенную библиотеку, гарантирующую быструю и удобную работу с двумерной графикой и обработкой пользовательского ввода. Благодаря аппаратному ускорению, она оптимально подходит для создаваемого приложения, визуализации макетной платы и реализации механики drag-and-drop, избавляя разработчика от написания избыточного низкоуровневого кода.

3 Язык программирования Lua используется в качестве встроенного скриптового движка для интерпретации и выполнения пользовательских программ непосредственно в процессе симуляции.

4 Библиотека sol2 является связующим звеном для интеграции Lua в код на C++. Она обеспечивает безопасный и удобный перенос данных, функций и объектов между движком симулятора и скриптовой средой исполнения.

1.3 Требования к приложению

Учитывая образовательную направленность продукта и необходимость наглядно знакомить пользователя с реальным устройством Arduino UNO, графический интерфейс должен быть оформлен в стиле, передающем внешний вид платы и радиокомпонентов. Главный экран будет разделён на рабочую область для сборки схем и текстовый редактор для ввода кода – это отразит два связанных этапа работы пользователя (сборка цепи и её программирование). Для управления симуляцией должны быть предусмотрены кнопки «Запустить», «Остановить», «Сбросить», как минимально достаточный набор для полного цикла работы с уровнем. Интерфейс должен остаться отзывчивым при любом разрешении экрана – для этого будет использоваться логическое разрешение 1920×1080 с автоматическим масштабированием области отображения под текущий размер окна с сохранением пропорций; отрисовка будет ограничена 60 кадрами в секунду, обработка событий мыши будет выполняться на каждом кадре без накопления задержки.

Так как пользователи только осваивают схемотехнику, ошибки с их стороны (короткое замыкание, синтаксически некорректный код) ожидаемы и не должны приводить к сбою программы – поэтому такие ситуации будут перехватываться и сопровождаться информационными окнами без зависания или закрытия приложения. Трансляция пользовательского кода будет вынесена в отдельный поток, чтобы выполнение потенциально некорректного скрипта не блокировало основной цикл отрисовки.

2 Проектирование

2.1 Организация данных

Проектирование задачи является важным и ответственным этапом разработки приложения, поскольку именно на этом этапе определяется структура данных и логика взаимодействия компонентов. От грамотной организации данных напрямую зависят производительность, масштабируемость и стабильность работы программы, особенно при покадровой эмуляции электрической цепи, где требуется непрерывный пересчёт состояния контактов и связей между ними.

В разрабатываемой программе используются три вида данных.

Первым видом являются данные, задаваемые разработчиком на этапе реализации. К ним относятся: координаты контактных точек платы, идентификаторы служебных выводов питания (5 В, 3,3 В, GND) и аналоговых выходов, заданные в виде локальных констант-массивов; тексты условий заданий уровней, заданные строковыми значениями подставляемые в зависимости от номера текущего уровня; графические ресурсы (текстуры) и файл шрифта, загружаемые при запуске программы и хранящиеся в объектах класса ResourceManager.

Вторым видом данных является динамическое состояние схемы и приложения: список контактов платы и радиодеталей, список проложенных пользователем проводов и логических соединений, граф связей между контактами (CircuitManager), номер активного уровня, а также флаги отображения окон, состояние выполнения пользовательского скрипта. Эти данные хранятся в полях структуры ArduinoSimulatorApp, в объектах классов Pin, CircuitManager и ComponentNetwork, а также в глобальных переменных (Globals.hpp).

Третьим видом данных являются результаты вычислений логики приложения, выполняемых на каждом кадре главного цикла и не сохраняемые постоянно: логический уровень каждого вывода (Floating/Low/High), факт обнаружения короткого замыкания; текущий угол поворота сервопривода, вычисляемый интерполяцией к целевому значению; результат проверки попадания клика мыши по контактам, проводам и элементам интерфейса. Эти значения пересчитываются заново на каждом кадре на основании текущей топологии схемы и пользовательского ввода.

Основной структурой хранения динамических объектов (проводов, соединений) выступает динамический массив std::vector. Выбор обусловлен необходимостью добавления новых проводов и контактов и их удаления в течение игровой сессии без заранее известного количества элементов.

2.2 Процессы

Весь процесс функционирования приложения строится на базе главного цикла, который непрерывно выполняет три основные фазы: обработку событий, обновление логики и отрисовку графики.

На этапе обновления состояния выполняются следующие действия:

- обход графа соединений в ширину для определения логического уровня каждого контакта;
- проверка условий для зажигания светодиода и плавного поворота вала сервопривода к целевому углу;
- обнаружение перегорания светодиода при отсутствии резистора и остановка выполнения скрипта при возникновении любой аварийной ситуации;
- выполнение пользовательского кода Lua-интерпретатором в отдельном потоке.

Каждый контакт обладает ролью и логическим состоянием, присваиваемым по результатам обхода графа, описанного выше.

Провод соединяет два контакта, прокладывается с привязкой к сетке и может быть удалён или перекрашен через контекстное меню.

Параллельно главному циклу в отдельном потоке (std::thread) интерпретатором Lua выполняются функции setup() и loop() пользовательского кода, которые изменяют состояния контактов виртуальной платы.

Для реализации перечисленных процессов использованы принципы объектно-ориентированного программирования [9]: каждая физическая сущность схемы и каждый логический узел приложения реализованы в виде отдельного класса со своим набором методов.

2.3 Описание внешнего пользовательского интерфейса

Важным аспектом при выполнении курсового проекта является организация интуитивно понятного диалога между пользователем и программой. Разработчик должен спроектировать внешний вид так, чтобы управление компонентами и написание кода не вызывало затруднений, а сам интерфейс соответствовал заявленной технической стилистике.

На рисунке 2.1 представлена схема навигации между окнами приложения. Переходы между экранами осуществляются через кнопки главного меню: «Теория», «Практика», «Настройки» и «Выход», также присутствует кнопка «Справка» для открытия справочного материала.

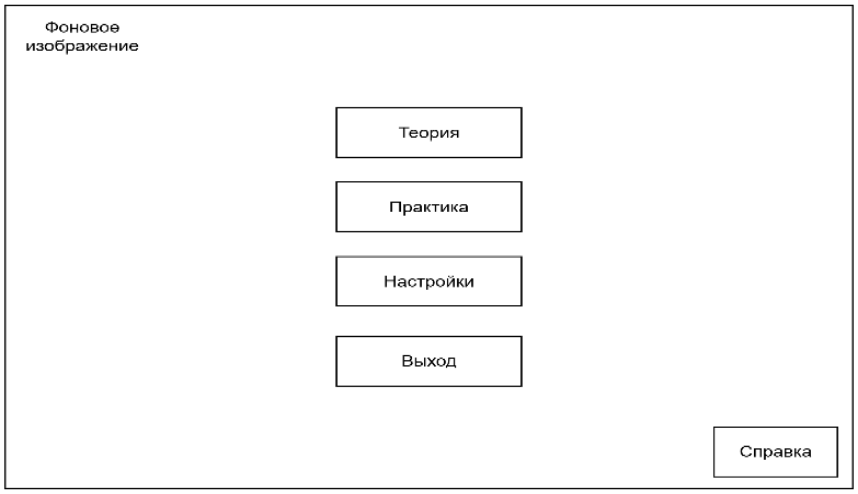


Рисунок 2.1 – Главное меню

На рисунке 2.2 представлена организация окна теории. Окно содержит карусель слайдов с теоретическим материалом. Навигация между слайдами осуществляется стрелками влево и вправо. Кнопка «Назад» выполняет возврат в главное меню.

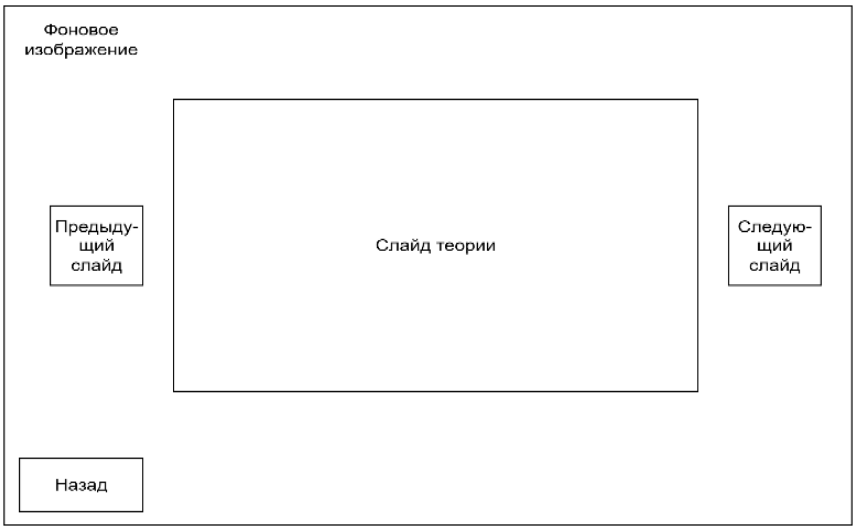


Рисунок 2.2 – Окно теории

На рисунке 2.3 представлена организация окна выбора уровня. Окно отображает карусель уровней с их названиями и описаниями. Кнопка «Начать» выполняет переход к выбранному уровню, кнопка «Назад» – возврат в главное меню.

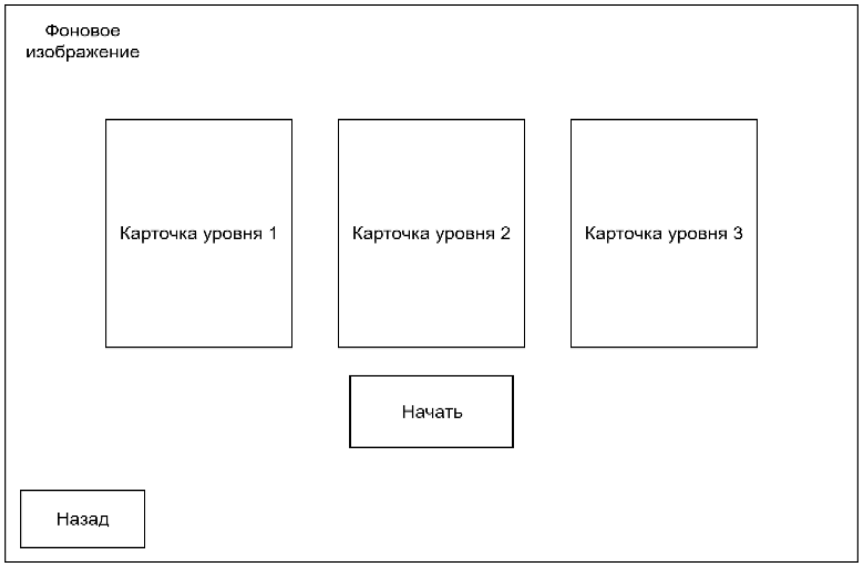


Рисунок 2.3 – Окно практики

На рисунке 2.4 представлена организация окна практики. Окно содержит текстовый редактор кода, область для сборки схемы, кнопки управления запуском («Запустить», «Остановить», «Сбросить»), а также кнопку отображения условия задачи.

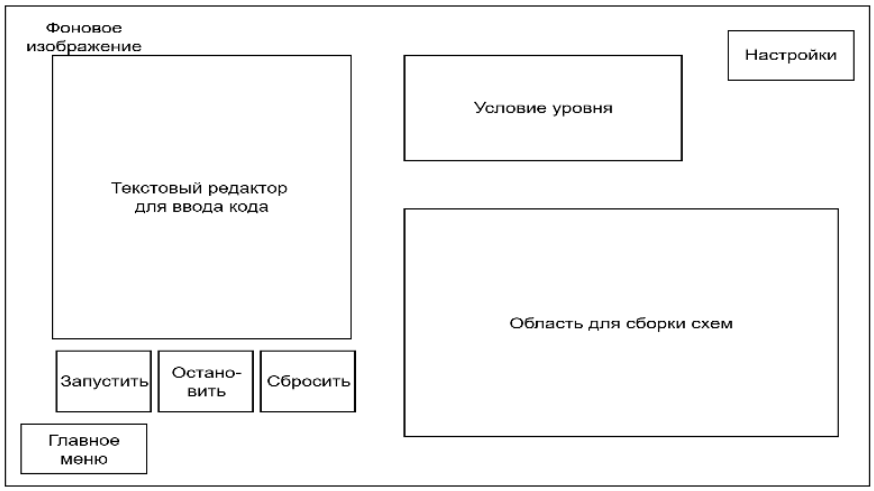


Рисунок 2.4 – Окно уровня

На рисунке 2.5 представлено всплывающее окно результата. Окно появляется поверх экрана практики при успешном прохождении уровня или при возникновении ошибки во время выполнения кода. Содержит соответствующее сообщение и кнопки перехода к следующему уровню, повтора или возврата в меню.

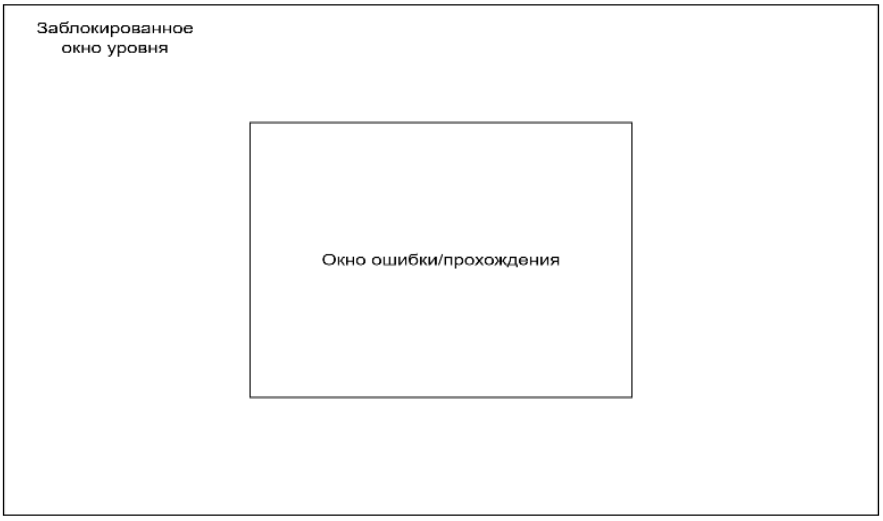


Рисунок 2.5 – Всплывающее окно сообщение

На рисунке 2.6 представлена организация окна настроек. Окно содержит кнопки переключения полноэкранного режима и включения или отключения звука, а также кнопку возврата в главное меню.

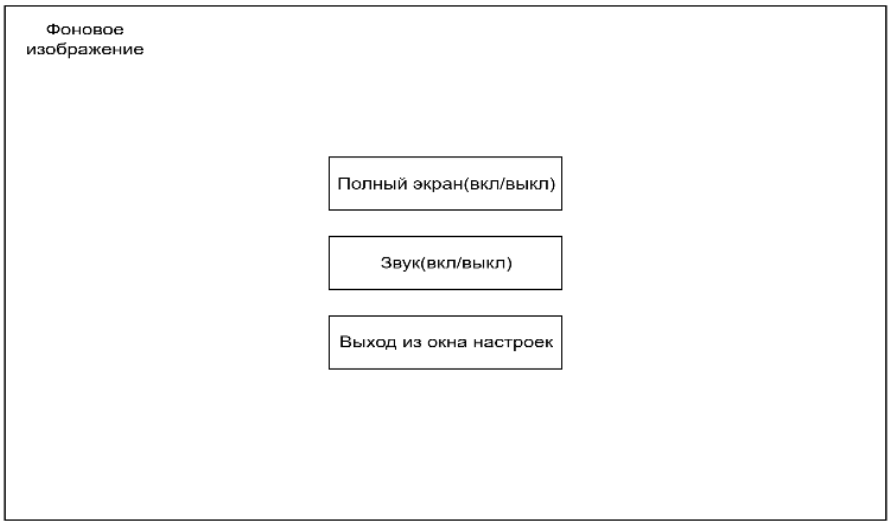


Рисунок 2.6 – Окно настроек

Таким образом, визуальное проектирование интерфейса обеспечивает соответствие всем техническим требованиям задания, исключает перегруженность графическими элементами и предоставляет пользователю комфортную среду для обучения. Сквозная интеграция программных модулей заставит все описанные компоненты работать как единое целое.

3 Реализация

3.1 Структура программы

Электронный образовательный ресурс по электронной платформе «Ардуино» содержит 13 основных модулей. Ниже рассмотрено назначение каждого модуля.

Модуль Main – точка входа в программу. Создаёт окно SFML, объект ArduinoSimulatorApp и запускает главный цикл.

Модуль ArduinoSimulatorApp – главная структура приложения. Управляет машиной состояний, хранит все игровые объекты текущего уровня, выполняет сброс схемы уровня и принудительную остановку пользовательского Lua-кода.

Модуль ResourceManager – управляет ресурсами приложения. Загружает и хранит текстуры, шрифт и звуковые эффекты, предоставляет методы воспроизведения звука.

Модули CircuitManager и ComponentNetwork – отвечают за логику электрической схемы. CircuitManager хранит граф соединений между контактами, ComponentNetwork обходит граф, пересчитывает логику выявляет короткое замыкание.

Модуль VirtualArduino – программная модель платы Arduino. Эмулирует функции Arduino API (pinMode, digitalWrite, digitalRead) для выполнения пользовательского Lua-кода в отдельном потоке, синхронизирует состояние с виртуальными контактами платы.

Модуль Pin – описывает контакт платы или радиодетали. Хранит координаты, роль и логическое состояние, обрабатывает клики пользователя при прокладке проводов.

Модуль Wire – описывает провод схемы. Класс Wire – один отрезок провода с привязкой к сетке, структура WireConnection объединяет один или несколько отрезков в полноценное соединение между двумя контактами.

Модуль Components – описывает виртуальные радиодетали макета: резистор, светодиод, физическую кнопку и сервопривод с анимацией поворота рога.

Модуль Buttons – элементы интерфейса. Класс Button – кнопка со спрайтом-текстурой, реагирует на наведение, клик и звук нажатия; TransparentButton – невидимая область клика поверх готовых изображений (окна ошибок, победы).

Модуль TextBox – многострочное текстовое поле для ввода и редактирования пользовательского Lua-кода: прокрутка, выделение.

Модуль ContextMenu – контекстное меню провода. Открывается по клику правой кнопкой мыши на проводе, позволяет удалить соединение или изменить его цвет.

Модуль LevelCarousel и TheoryCarousel – карусели изображений. LevelCarousel – карточки уровней на экране выбора с прокруткой и масштабированием, TheoryCarousel – слайды теоретического материала по электронике и Arduino.

Модуль Globals / Helpers / Utils / Common – вспомогательные модули. Globals хранит общие для приложения переменные состояния, Helpers – общие утилиты, Utils – разметку контактов платы и генерацию условия уровня, Common –подключение библиотек и общие перечисления.

3.2 Структура и описание процедур и функций пользователя

Описание разработанных процедур и функций для программного продукта приведено в таблице 1.

Таблица 1 – Процедуры и функции

Имя процедуры (функции)	В каком модуле находится	За каким элементом управления закреплена	Назначение
1	2	3	4
resetSimulationEnvironment()	ArduinoSimulatorApp	btnSimResetTask и другие кнопки возврата к уровню	Возвращает схему, провода и состояние платы к начальному состоянию текущего уровня
forceStopCode()	ArduinoSimulatorApp	stopButton	Принудительно останавливает фоновый поток выполнения пользовательского Lua-скетча
createAutoWire()	ArduinoSimulatorApp	Вызывается из resetSimulationEnvironment()	Строит графическое и логическое соединение (провод) между двумя контактами при инициализации уровня
ComponentNetwork::resolveLogic()	ComponentNetwork	Вызывается при изменении схемы	Пересчитывает логические уровни всей цепи и выявляет короткое замыкание

Продолжение таблицы 1

1	2	3	4
VirtualArduino::pinMode() / digitalWrite() / digitalRead()	VirtualArduino	Вызываются из Lua-скрипта пользователя	Эмулируют функции Arduino API, связывая код с виртуальными контактами платы
ContextMenu::handleLeftClick()	ContextMenu	Пункты контекстного меню провода	Обрабатывает удаление провода или изменение его цвета
generateLevelTask()	Utils	Вызывается при входе/смене уровня	Случайным образом генерирует условие задания (номера пинов платы)
add_pins() / getPinLabel()	Utils	Инициализация платы	Размечает координаты пинов платы и формирует их подписи
Wire::snapToGrid() / search_angle()	Wire	Прокладка провода пользователем	Привязывает конец провода к сетке по горизонтали/вертикали
CircuitManager::addConnection() / removeConnection()	CircuitManager	Клик по пинам / удаление провода	Добавляет или удаляет ребро в графе соединений схемы
Button::checkClick() / TransparentButton::checkClick()	Buttons	Клик по UI-кнопке	Проверяет попадание клика в область кнопки и воспроизводит звук нажатия

3.3 Описание использованных компонентов

Описание использованных при разработке компонентов для программного продукта – электронного образовательного ресурса по электронной платформе «Ардуино» приведено в таблице 2.

Таблица 2 – Использованные компоненты

Компонент	Где используется	Назначение
1	2	3
sf::RenderWindow	main.cpp	Главное графическое окно приложения, отрисовка всех экранов

Продолжение таблицы 2

1	2	3
sf::Texture, sf::Sprite	ResourceManager, Components, Buttons	Загрузка и отображение изображений радиодеталей, платы, кнопок и фонов
sf::SoundBuffer, sf::Sound	ResourceManager	Загрузка и воспроизведение звуковых эффектов интерфейса
sf::Font, sf::Text	ResourceManager, TextBox, UI-экраны	Загрузка шрифта и отображение текстовых элементов
sf::RectangleShape, sf::CircleShape	Wire, Buttons, Pin	Отрисовка провода, кнопок и контактов схемы
sf::Event	main.cpp	Обработка пользовательских событий
sf::Clock	main.cpp, ArduinoSimulatorApp	Измерение времени кадра, таймеры уведомлений и анимаций
sol::state (библиотека sol2 / Lua)	ArduinoSimulatorApp, VirtualArduino	Встроенный интерпретатор Lua; выполняет пользовательский код и вызывает функции VirtualArduino
std::vector, std::unordered_map, std::unordered_set	Окно приложения	Хранение списков контактов, проводов, графа соединений и состояний логики
std::thread, std::mutex, std::atomic	Экран симуляции	Выполнение Lua-скрипта в отдельном потоке без блокировки интерфейса, безопасный обмен данными между потоками

3.4 Спецификация программы

Точное техническое наименование разработанного проекта, состав файлов исходного кода, задействованных ресурсов и результирующих наборов данных приведены в таблице 3.

Таблица 3 – Спецификация программы

Имя файла	Назначение
1	2
ArduinoSimulator.exe	Исполняемый файл проекта, используется для запуска программы в ОС Windows
main.cpp	Точка входа, главный игровой цикл, обработка событий и отрисовка кадра
Common.hpp	Подключение библиотек (SFML, sol2, STL), перечисления GameState, LogicLevel, PinRole

Продолжение таблицы 3

1	2
Globals.hpp/ .cpp	Глобальные переменные состояния приложения
Helpers.hpp / .cpp	Вспомогательные функции: replaceAll, getDistanceToSegment
Utils.hpp / .cpp	Разметка пинов платы и генерация условия уровня
ArduinoSimulatorApp.hpp / .cpp	Главная структура приложения: машина состояний, игровые объекты уровня, сброс/остановка
ResourceManager.hpp / .cpp	Загрузка текстур, шрифта и звуковых эффектов; их воспроизведение
Pin.hpp / .cpp	Контакт платы или радиодетали: координаты, роль, логическое состояние
CircuitManager.hpp / .cpp	Граф соединений схемы: добавление/удаление проводов
ComponentNetwork.hpp / .cpp	Расчёт логических уровней схемы, поиск короткого замыкания
VirtualArduino.hpp / .cpp	Программная модель платы Arduino для выполнения Lua-кода
Buttons.hpp / .cpp	Классы Button и TransparentButton – кнопки интерфейса
Wire.hpp / .cpp	Класс Wire (отрезок провода) и структура WireConnection (соединение)
Components.hpp / .cpp	Классы радиодеталей: LED, Resistor, ServoComponent, ArduinoButton
LevelCarousel.hpp / .cpp	Карусель карточек уровней на экране выбора
TheoryCarousel.hpp / .cpp	Карусель слайдов теоретического материала
TextBox.hpp / .cpp	Многострочное текстовое поле ввода и редактирования кода
ContextMenu.hpp / .cpp	Контекстное меню провода: удаление, смена цвета
icon.ico, icon.rc	Иконка приложения и ресурсный скрипт Windows для её подключения
sounds/*.mp3	Звуковые файлы
images/buttons/*.png	Текстуры кнопок
images/components/*.png	Текстуры платы и радиодеталей
images/levels/*.png	Текстуры карточек уровней
images/theory/*.jpg	Текстуры слайдов теории
undertale_battle_font.ttf	Используемый шрифт
help.chm	Файл справочной системы

4 Тестирование

В процессе создания программного комплекса возникающие ошибки и недоработки устранялись непосредственно на этапе реализации проекта. После завершения отладки основных модулей было выполнено комплексное функциональное тестирование. Проведение проверки призваны подтвердить корректное и автономное функционирование всех компонентов приложения.

В этом разделе представлен отчет по итогам проведенных испытаний. Подробные результаты проверок сведены в таблицу 4.

Таблица 4 – Отчет о результатах тестирования

Тест	Ожидаемый результат	Фактический результат	Результат тестирования
1	2	3	4
Проверка запуска приложения	Отобразится экран загрузки с последующим автоматическим переходом в главное меню и отрисовкой графического интерфейса	Отобразился экран загрузки с последующим автоматическим переходом в главное меню и отрисовкой графического интерфейса	Выполнено
Проверка кнопки «Теория» главного меню	Перейдет в соответствующий раздел, позиция карусели сбросится на первый слайд	Перешло в соответствующий раздел, позиция карусели сбросилась на первый слайд	Выполнено
Проверка навигации по слайдам теории (пролистать слайды кнопками на экране, клавишами ←/→ и колесом мыши.)	Слайды поменяются во всех трех случаях, на первом слайде не будет доступна кнопка «предыдущий слайд», на последнем – «следующий слайд»	Слайды поменялись во всех трех случаях, на первом слайде не была доступна кнопка «предыдущий слайд», на последнем – «следующий слайд»	Выполнено
Проверка кнопки «Назад» в разделе теории	Вернется в главное меню	Вернулось в главное меню	Выполнено

Продолжение таблицы 4

1	2	3	4
Проверка кнопки «Настройки» главного меню (нажать на кнопку «Настройки», переключить режимы экрана и звука, нажать на кнопку «Назад»)	Перейдет в раздел настроек, окно корректно переключится между оконным и полноэкранным режимом, переключится режим работы звука, вернется в главное меню с сохранением настроек	Перешло в раздел настроек, окно корректно переключилось между оконным и полноэкранным режимом, переключился режим работы звука, вернулся в главное меню с сохранением настроек	Выполнено
Проверка кнопки «Практика» главного меню	Откроется экран выбора уровня, позиция карусели сбросится на первый уровень	Открылся экран выбора уровня, позиция карусели сбросилась на первый уровень	Выполнено
Проверка кнопки «Назад» из раздела практики	Вернется в главное меню	Вернулось в главное меню	Выполнено
Проверка запуска уровня (нажать на кнопку «Начать», дважды кликнуть по карточке уровня)	Сгенерируется задание, выполниться переход в режим симуляции, отобразиться условие задания	Сгенерировался уровень, выполнился переход в режим симуляции, отобразилось условие задания	Выполнено
Проверка кнопки скрытие/отображения условия задания	По первому нажатию условие скроется, по повторному – отобразиться	По первому нажатию условие скрылось, по повторному – отобразилось	Выполнено
Проверка раздела настроек в режиме симуляции (нажать на кнопку «Настройки», переключить режимы экрана и звука)	Переключения должны работать также, как в главном меню, без выхода из уровня и без сброса текущей схемы	Переключения сработали также, как в главном меню, без выхода из уровня и без сброса текущей схемы	Выполнено

Продолжение таблицы 4

1	2	3	4
Проверка создания провода между пинами (кликнуть по одному из пинов, кликнуть по второму, к которому нужно подключиться)	Между пинами построиться графический провод, в массиве логических соединений появиться соответствующая связь	Между пинами построился графический провод, в массиве логических соединений появилась соответствующая связь	Выполнено
Проверка контекстного меню провода (кликнуть правой кнопкой мыши по проведенному пользователем проводу)	Откроется меню рядом с проводом, выбранное действие изменит цвет или удалит соединение графически и логически	Открылось меню рядом с проводом, выбранное действие изменило цвет или удалило соединение графически и логически	Выполнено
Проверка компиляции пользовательского кода с синтаксической ошибкой	Перехватится исключение, запуск отклонится, отобразится окно «Требуется точка с запятой» и воспроизведется звук ошибки	Перехватилось исключение, запуск отклонился, отобразилось окно «Требуется точка с запятой» и воспроизвелся звук ошибки	Выполнено
Проверка защиты светодиода от превышения напряжения (подключить светодиод напрямую к 5 вольтам, минуя резистор)	Светодиод «перегорит», скрипт принудительно остановится, отобразится окно «Светодиод сгорел» и воспроизведется звук ошибки	Светодиод «перегорел», скрипт принудительно остановился, отобразилось окно «Светодиод сгорел» и воспроизвелся звук ошибки	Выполнено

Продолжение таблицы 4

1	2	3	4
Проверка фиксации короткого замыкания (соединить проводом вывод 5 вольт напрямую с землей, минуя радиодетали)	Обнаружится короткое замыкание, выполнение схемы остановится, отобразится окно «Плата сгорела», воспроизведется звук ошибки	Обнаружилось короткое замыкание, выполнение схемы остановилось, отобразилось окно «Плата сгорела», воспроизвелся звук ошибки	Выполнено
Проверка кнопки «Стоп» (запустить пользовательский код, нажать кнопку «Стоп»)	Поток выполнения скрипта корректно остановиться, все пины сбросятся состояние (кроме констант), отобразиться уведомление «Код остановлен», воспроизведется звук остановки	Поток выполнения скрипта корректно остановился, все пины сбросили состояние (кроме констант), отобразилось уведомление «Код остановлен», воспроизвелся звук остановки	Выполнено
Проверка условия прохождения уровня (собрать схему, написать и запустить код, выполняющий задание текущего уровня)	Скрипт остановится, отобразится окно победы с возможностью перехода к следующему уровню или перезапуску текущего, воспроизведется звук победы	Скрипт остановился, отобразилось окно победы с возможностью перехода к следующему уровню или перезапуску текущего, воспроизвелся звук победы	Выполнено

Продолжение таблицы 4

1	2	3	4
Проверка кнопки «Сбросить» (проложить один или несколько проводов и/или изменить код, нажать на кнопку «Сбросить»)	Все провода, добавленные пользователем, удалятся, текст кода (для уровней с редактируемым кодом) вернуться к исходному шаблону, индикаторы ошибок сбросятся	Все провода, добавленные пользователем, удалились, текст кода (для уровней с редактируемым кодом) вернулись к исходному шаблону, индикаторы ошибок сбросились	Выполнено
Проверка кнопки «Выход»	Приложение корректно закроется, занятая оперативная память освободится	Приложение корректно закрывается, занятая оперативная память освобождается	Выполнено

Во время создания программного продукта был успешно устранен ряд технических трудностей, в частности: некорректное определение точки привязки проводов к пинам платы и компонентов при построении трассы, полное зависание интерфейса при начале выполнения Lua-скрипта (пользовательского кода), а также некорректная обработка ошибок и состояния пинов. Благодаря выполненным исправлениям конечный пользователь не столкнётся с подобными неполадками.

Все модули приложения прошли проверку, в ходе которой было установлено, что они работают стабильно и корректно выполняют возложенные на них функции.

5 Руководство пользователя

5.1 Общие сведения о программном продукте

Цель разработанного проекта состоит в обучении пользователя конструированию электрических цепей, принципам работы микроконтроллера Arduino UNO и особенностям написания управляющего кода.

Разработанная виртуальная лаборатория ориентирована на широкую аудиторию, интересующуюся робототехникой, схемотехникой и безопасным тестированием алгоритмов без риска повреждения физической аппаратуры.

Производительность программного обеспечения во многом зависит от аппаратных характеристик персонального компьютера: тактовой частоты процессора, объема оперативной памяти. Несмотря на сложность эмуляции процессов, симулятор быстро запускается и стабильно функционирует на современных вычислительных машинах благодаря оптимизации кода на C++ и графической библиотеке SFML, которая для корректной отрисовки требует поддержки OpenGL 3.3. Запуск исполняемого файла предполагает наличие установленного пакета Visual C++ Redistributable и 64-разрядную ОС. Так как симулятор рассчитан на одиночное использование без сетевого взаимодействия, все вычисления логики микроконтроллера и электрических параметров выполняются локально, в двумерном пространстве.

Тестирование проводилось на различных конфигурациях ЭВМ, и взаимодействие с программным продуктом было комфортным. Разработка и отладка велась на ПК со следующими характеристиками.

- процессор AMD Ryzen 7 7735HS, базовая частота 3.20 ГГц, максимальная частота в турбо-режиме до 4.75 ГГц, 8 ядер, 16 логических потоков;
- объем ОЗУ 16 ГБ, тип DDR5;
- графический адаптер дискретная видеокарта NVIDIA GeForce RTX 4060m;
- операционная система Windows 11 Home.

5.2 Инсталляция

Для установки приложения необходимо запустить исполняемый файл ARDUINO SIMULATOR_installer.exe, представленный на рисунке 5.1. На экране появится стартовое окно мастера инсталляции электронного образовательного ресурса по электронной платформе «Ардуино». Далее требуется следовать инструкциям программы установки, указав целевую директорию для распаковки системных файлов и ресурсов приложения.

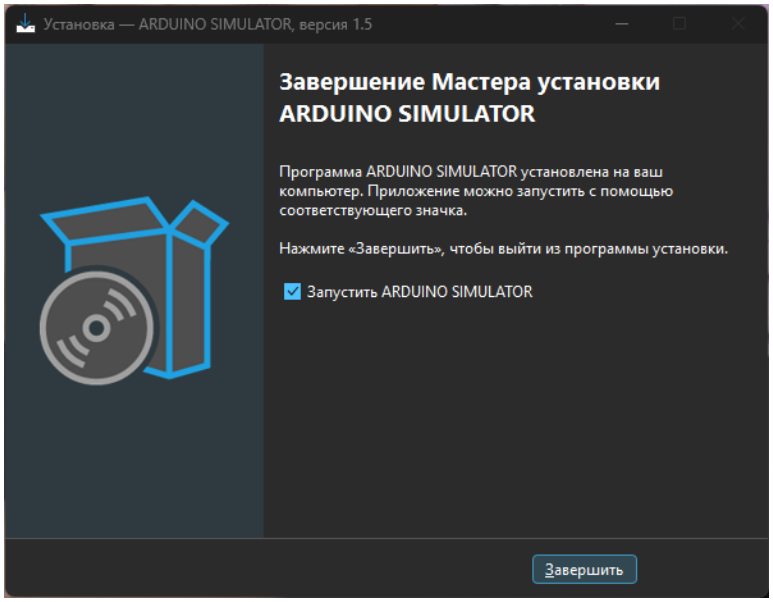


Рисунок 5.1 – Завершение установки приложения

5.3 Выполнения программы

5.3.1 Запуск программы

Запустить электронный ресурс можно несколькими способами.

Первым вариантом является использование ярлыка на рабочем столе. Требуется дважды кликнуть левой кнопкой мыши по иконке с названием «ARDUINO SIM».

Вторым способом выступает прямой запуск исполняемого файла из корневого каталога, выбранного при установке (по умолчанию C:\ArduinoSimulator).

5.3.2 Инструкции по работе с программой

После запуска на экране нас отображается главное меню приложения, представленного на рисунке 5.2.



Рисунок 5.2 – Главное меню

Навигация в стартовом окне осуществляется посредством нескольких элементов управления, каждый из которых отвечает за определенный модуль симулятора.

При нажатии на кнопку «Теория» открывается встроенная презентация, предоставленная на рисунке 5.3. В ней содержится структурированный материал, необходимый для изучения основ схемотехники и принципов работы микроконтроллеров.



Рисунок 5.3 – Встроенная презентация с теорией

Кнопка «Практика» осуществляет переход к форме выбора доступных практических уровней, представленных на рисунке 5.4. После определения конкретной задачи запускается интерактивная сессия, где пользователь может принять полученные знания: собирать электрические цепи на виртуальной макетной плате и тестировать управляющий алгоритм, представленный на рисунке 5.5, на котором также можно заметить кнопки «Запустить» – благодаря ней запускается процесс, созданный пользователем, «Остановить» – с помощью которой можно остановить компиляцию, «Сбросить» – можно использовать преждевременно заметив ошибку, устранить ее, начав соединение проводов или написание кода заново, «В главное меню» – вернуться, если вы ошибочно выбрали уже пройденный уровень, «Условие» – можно прочесть конкретно заданное задание, указанное в уровне, «Настройки» – здесь можно увидеть все пункты, описанные ниже.



Рисунок 5.2 – Меню выбора уровня



Рисунок 5.5 – Выполнение задания выбранного уровня

Для регулировки параметров среды реализована кнопка «Настройки». При переходе в этот раздел открывается дополнительное меню, представленное на рисунке 5.6, состоящее из трех элементов.

- «Полный экран» – масштабирует рабочее окно на весь монитор;
- «Выключить звук» – деактивирует фоновое аудиосопровождение;
- «Назад» – возвращает пользователя к предыдущему экрану.

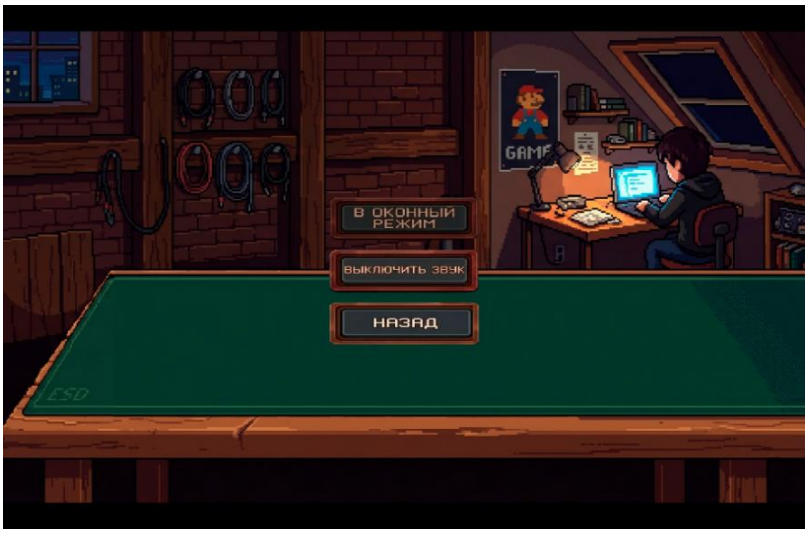


Рисунок 5.6 – Меню настроек

5.3.3 Завершение работы с программой

Завершение работы с приложением выполняется несколькими методами.

– нажатием на стандартный системный элемент (крестик) в правом верхнем углу окна;

– использованием специальной интерактивной кнопки «Выход», расположенной в центре главного меню.

5.4 Использование системы справочной информации

Вызов встроенной документации происходит при нажатии на кнопку «Справка», находящуюся в правом нижнем углу стартового экрана. Открывающийся раздел содержит краткую инструкцию с правилами использования программного обеспечения, а также базовые сведения об авторе проекта, предоставленных на рисунке 5.7.

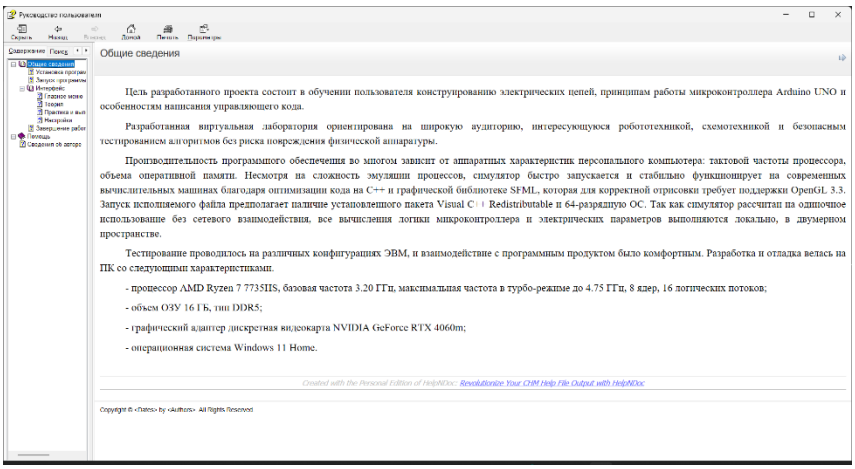


Рисунок 5.7 – Справочная система

Заключение

Цель курсового проекта заключалась в разработке программного продукта, предназначенного для обучения пользователей основам схемотехники и программирования микроконтроллеров через сборку электрических цепей на макетной плате и написание управляющего кода.

Разработанное приложение рассчитано на широкую аудиторию: школьников, студентов и любых энтузиастов, самостоятельно изучающих робототехнику. Программа избавляет от необходимости покупать дорогостоящее оборудование на начальных этапах обучения – пользователь может отрабатывать схемы и алгоритмы в безопасной виртуальной среде, в рамках предусмотренных учебных уровней.

В ходе выполнения курсовой работы был написан и протестирован программный комплекс с понятным интерфейсом. Задача выполнена в полном объеме: реализованы рабочее пространство для сборки схем, набор учебных уровней с собственным заданием для каждого, текстовый редактор пользовательского кода, логическая эмуляция электрической цепи и система диагностики типовых ошибок (синтаксическая ошибка кода, короткое замыкание, перегорание светодиода).

При разработке использовался объектно-ориентированный подход – он лучше всего подходит для гибкой и расширяемой архитектуры приложения. Язык C++ и библиотека SFML обеспечили высокую скорость вычислений и стабильную отрисовку компонентов без задержек. Для выполнения пользовательского кода интегрирована библиотека sol2, связывающая интерпретатор Lua с классами симулятора на C++.

Принятые проектные решения привели к тому, что все реализованные модули работают корректно и в полной мере соответствуют поставленному заданию. Для эмуляции схемы использована графовая модель: любое изменение схемы запускает обход графа соединений и пересчёт логического состояния каждого узла. Это позволяет получить отклик компонентов в режиме, близком к реальному времени, без точных физических расчётов токов и напряжений.

В качестве вектора дальнейшего развития возможна доработка приложения с целью расширения набора доступных компонентов (добавление новых датчиков, дисплеев), внедрения виртуального монитора последовательного порта и измерительных приборов (мультиметра), реализации сохранения и загрузки собранных пользователем схем, а также перехода от логической модели цепи к расчётной, основанной на законе Ома, для более точной имитации реальных номиналов компонентов.

Разработанный программный продукт является законченной, работоспособной программой, готовой к практическому применению в образовательных целях.

Список использованных источников

1. Официальная документация платформы Arduino: справочник по аппаратному и программному обеспечению (использовано для построения логики работы микроконтроллера и эмуляции электронных компонентов) [Электронный ресурс]. – Режим доступа: https://docs.arduino.cc/ – Дата доступа: 12.06.2026;
2. Официальная документация библиотеки sol2 [Электронный ресурс]. – Режим доступа: https://sol2.readthedocs.io/ – Дата доступа: 15.06.2026;
3. Официальный репозиторий Simple and Fast Multimedia Library (SFML) [Электронный ресурс]. – Режим доступа: https://github.com/SFML/SFML – Дата доступа: 10.06.2026;
4. Прата, С. Язык программирования C++. Лекции и упражнения / С. Прата. – 6-е изд. – М.: Вильямс, 2012. – 1248 с;
5. Шилдт, Г. C++: базовый курс / Г. Шилдт. – 3-е изд. – М.: Вильямс, 2015. – 624 с;
6. Лафоре, Р. Объектно-ориентированное программирование в C++ / Р. Лафоре. – 4-е изд. – СПб.: Питер, 2018. – 928 с.
7. Официальный сайт языка программирования Lua [Электронный ресурс]. – Режим доступа: https://www.lua.org/ – Дата доступа: 16.06.2026;
8. Официальная документация API кроссплатформенной библиотеки SFML [Электронный ресурс]. – Режим доступа: https://www.sfml-dev.org/learn.php – Дата доступа: 14.06.2026;
9. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – СПб.: Питер, 2015. – 368 с. (Использовано для проектирования модульной архитектуры программы и паттернов взаимодействия физических объектов).